

# Cheatsheet — conversioni per il mapping (CALC)

Da tenere aperto mentre si studia e da rifare a memoria prima della prova: all'esame non si porta niente (dispensa 01 p. 8). Ogni regola ha un esempio già verificato. Nasce dalla sessione del 2026-10-08 (lezione [[lezione\_02p\_fondamenta]]); le regole di conto ridotte all'osso sono anche in [[prontuario\_CALC]] §0.

## 1. Potenze di due da sapere a memoria

| $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | $2^5$ | $2^6$ | $2^7$ | $2^8$ | $2^9$ | $2^{10}$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|
| 1     | 2     | 4     | 8     | 16    | 32    | 64    | 128   | 256   | 512   | 1024     |

**Le unità:** ogni gradino moltiplica per 1024, cioè **aggiunge 10 all'esponente**.

| K (kilo) | M (mega) | G (giga) |
|----------|----------|----------|
| $2^{10}$ | $2^{20}$ | $2^{30}$ |

## 2. Da una taglia ai bit interni (K)

**Regola:** spezza in *numero* × *unità*, scrivi entrambi come potenza di due, **somma gli esponenti**.

$$\begin{aligned} 32 \text{ KB} &= 2^5 \times 2^{10} = 2^{15} && \rightarrow K = 15 \\ 512 \text{ KB} &= 2^9 \times 2^{10} = 2^{19} && \rightarrow K = 19 \\ 64 \text{ MB} &= 2^6 \times 2^{20} = 2^{26} && \rightarrow K = 26 \\ 2 \text{ GB} &= 2^1 \times 2^{30} = 2^{31} && \rightarrow K = 31 \end{aligned}$$

- **K = numero di piedini d'indirizzo del chip** ( $A[K-1..0]$ ).
- **32 – K = bit che scelgono il blocco** (nel DLX, che ha 32 bit). 2 GB → resta 1 bit: BA31.

⚠ Non dimenticare l'unità: «32 KB → 5» è sbagliato, si scrivono **sempre due** potenze.

## 3. Da binario a decimale

**Regola:** ogni posizione vale il doppio di quella alla sua destra. Somma i pesi dove c'è 1.

$$\begin{array}{rcll} \text{pesi:} & 8 & 4 & 2 & 1 \\ & 1 & 1 & 0 & 0 & \rightarrow & 8 + 4 & = & 12 \\ & 1 & 1 & 0 & 1 & \rightarrow & 8 + 4 + 1 & = & 13 \end{array}$$

$$\begin{array}{ccccccccc} 16 & 8 & & 4 & & 2 & & 1 & \\ 1 & 0 & & 1 & & 1 & & 0 & \end{array} \rightarrow 16 + 4 + 2 = 22$$

**Controllo:** rifai la somma partendo da destra. 1100 non è 8: c'è anche il 4.

---

#### 4. Da decimale a binario

**Regola:** togli la potenza di due più grande che ci sta, metti 1 in quella posizione, ripeti col resto. Dove non togli niente, metti 0.

12: ci sta 8 → resto 4; ci sta 4 → resto 0 → 8 4 2 1 = 1 1 0 0  
 6: non ci sta 8; ci sta 4 → resto 2; ci sta 2 → 0 → 8 4 2 1 = 0 1 1 0

---

#### 5. Esadecimale ↔ binario

**Una cifra esadecimale = 4 bit esatti.** La h in fondo vuol dire «esadecimale».

|        |        |        |        |
|--------|--------|--------|--------|
| 0 0000 | 4 0100 | 8 1000 | C 1100 |
| 1 0001 | 5 0101 | 9 1001 | D 1101 |
| 2 0010 | 6 0110 | A 1010 | E 1110 |
| 3 0011 | 7 0111 | B 1011 | F 1111 |

Trucco per ricostruire la tabella: A = 10, B = 11, ... F = 15, e poi la conversione del §4.

**Da hex a binario:** sostituisci ogni cifra con i suoi 4 bit.

2800h = 2 8 0 0 = 0010 1000 0000 0000  
 C000h = C 0 0 0 = 1100 0000 0000 0000

**Da binario a hex:** dividi in gruppi di 4 **partendo da destra**, poi converti ogni gruppo.

0010 0111 1111 1111 = 2 7 F F = 27FFh

**Indirizzi del DLX:** 32 bit = **8 cifre** hex. Il bit più a sinistra, BA31, è il primo bit della prima cifra.

80000000h = 1000 0000 ... 0000 → solo BA31 = 1  
 40000000h = 0100 0000 ... 0000 → solo BA30 = 1

---

#### 6. Le taglie in esadecimale (DLX)

La taglia scritta in hex è **anche l'indirizzo dove finisce il blocco che parte da 0, più uno.**

| Taglia | Potenza  | In hex    |
|--------|----------|-----------|
| 1 MB   | $2^{20}$ | 00100000h |
| 16 MB  | $2^{24}$ | 01000000h |

| Taglia | Potenza  | In hex    |
|--------|----------|-----------|
| 64 MB  | $2^{26}$ | 04000000h |
| 128 MB | $2^{27}$ | 08000000h |
| 256 MB | $2^{28}$ | 10000000h |
| 512 MB | $2^{29}$ | 20000000h |
| 1 GB   | $2^{30}$ | 40000000h |
| 2 GB   | $2^{31}$ | 80000000h |

**Come ricavarle senza tabella:** dividi l'esponente per 4. Il **quoziente** è il numero di zeri in fondo, il **resto** dice la cifra davanti (resto 0 → 1, 1 → 2, 2 → 4, 3 → 8).

$2^{30}$ :  $30 = 4 \times 7 + 2 \rightarrow$  cifra 4, poi 7 zeri → 40000000h

$2^{26}$ :  $26 = 4 \times 6 + 2 \rightarrow$  cifra 4, poi 6 zeri → 04000000h (8 cifre: si riempie a sinistra con

## 7. Fine di un blocco

**Regola:** fine = inizio + taglia – 1. In binario: gli **ultimi K bit tutti a 1**.

1 GB da 00000000h:  $00000000h + 40000000h - 1 = 3FFFFFFh$

512 MB da 40000000h:  $40000000h + 20000000h - 1 = 5FFFFFFh$

64 MB da 60000000h:  $60000000h + 04000000h - 1 = 63FFFFFFh$

1 GB da 80000000h:  $80000000h + 40000000h - 1 = BFFFFFFh$

Il «– 1» c'è perché si conta **da 0**: 4 celle sono 0, 1, 2, 3, e l'ultima è la 3.

## 8. Allineamento: il blocco può partire da qui?

**Regola:** un blocco da  $2^K$  parte da un indirizzo allineato se **gli ultimi K bit sono 0**, cioè se l'indirizzo è un multiplo della taglia.

Scorciatoie (dispensa 02 p. 38): - da 2 byte: indirizzo **pari** (ultimo bit 0); - da 8 byte: ultimi **3** bit a 0; - da 16 byte: ultima **cifra hex** a 0; - in generale, se K è multiplo di 4: le ultime **K/4 cifre hex** a 0.

64 MB (K = 26) a 60000000h?  $60000000h = 0110\ 0000 \dots$ : dopo il bit 29 è tutto 0 → sì  
512 MB (K = 29) a 04000000h?  $04000000h = 0000\ 0100 \dots$ : il bit 26 è 1, sotto i 29 → no

**Per non sbagliare:** disponi i blocchi **dal più grande al più piccolo** partendo da 0, e l'allineamento viene da solo (lezione 02, Decisione 1).

## 9. Bit alti e bit bassi

«Alti» = le cifre **a sinistra** nella scritta del numero, che pesano di più. «Bassi» = quelle **a destra**.

- i **K bit bassi** scelgono la **cella** dentro il chip (vanno ai piedini A[K-1..0]);
- i bit **alti** che restano scelgono **quale chip**: sono quelli che usi nel CS.

esempio con 3 bit e chip da 4 celle (K = 2):

|      |  |       |   |
|------|--|-------|---|
| 1    |  | 0     | 1 |
| chip |  | cella |   |

## 10. Conti di mapping in hex (bus a 16 bit)

Aggiunta il 2026-10-09, dall'esercizio 10 della dispensa (lezione 02p §6). Sono le quattro operazioni che si ripetono in ogni mappa.

**a. Le taglie a 16 bit, a memoria.** Raddoppiando, la cifra raddoppia: 4 → 8 → 1(0) → 2 → 4 → 8.

| 1K    | 2K    | 4K    | 8K    | 16K   | 32K   |
|-------|-------|-------|-------|-------|-------|
| 0400h | 0800h | 1000h | 2000h | 4000h | 8000h |

**b. Inizio + taglia = inizio del pezzo dopo.** Si somma cifra per cifra; il riporto scatta a 16, non a 10.

|              |              |                                                 |
|--------------|--------------|-------------------------------------------------|
| 2000h        | 2800h        |                                                 |
| + 0800h (2K) | + 0800h (2K) |                                                 |
| -----        | -----        |                                                 |
| 2800h        | 3000h        | ← 8 + 8 = 16 = «10» in hex: scrivi 0, riporta 1 |

**c. Fine = inizio del pezzo dopo – 1.** Gli zeri in fondo diventano **F**, la cifra prima scende di 1.

2800h – 1 = 27FFh      3000h – 1 = 2FFFh      C000h – 1 = BFFFh

**d. Allineamento quando K non è multiplo di 4.** Si guarda la cifra «a metà».

| Taglia                | Bit bassi a 0 | In hex                                           |
|-----------------------|---------------|--------------------------------------------------|
| 2K = 2 <sup>11</sup>  | 11            | ultime 2 cifre 00, la terza solo <b>0 o 8</b>    |
| 4K = 2 <sup>12</sup>  | 12            | ultime 3 cifre 000                               |
| 8K = 2 <sup>13</sup>  | 13            | ultime 3 cifre 000, la quarta <b>pari</b>        |
| 16K = 2 <sup>14</sup> | 14            | ultime 3 cifre 000, la quarta <b>0, 4, 8 o C</b> |

**Dove stanno i bit** (16 bit = 4 cifre):

|          |         |        |        |
|----------|---------|--------|--------|
| C        | 0       | 0      | 0      |
| A15..A12 | A11..A8 | A7..A4 | A3..A0 |

A13 = secondo bit della prima cifra; A11 = primo bit della seconda.

**e. Colonne con lettere, e il prestito a 16** (sessione del 2026-10-09). La h è un'etichetta, non una cifra: nel conto non entra. Ogni colonna contiene **una cifra sola**: si traduce la lettera (A = 10 ... F = 15), si fa il conto, e un risultato da 10 a 15 si ritraduce in lettera.

|       |         |                                                                                   |
|-------|---------|-----------------------------------------------------------------------------------|
| 1ACh  | 10000h  | (64K: inizio della EPROM da 16K in alto)                                          |
| + 27h | - 4000h |                                                                                   |
| ----  | -----   |                                                                                   |
| 1D3h  | 0C000h  | ← 0 - 4: prestito di 16 → 16 - 4 = 12 = C (non «12»);<br>l'1 che presta diventa 0 |

C + 7 = 12 + 7 = 19 ≥ 16 → scrivi 3, riporta 1

Verifica: C000h + 4000h = 10000h (C + 4 = 16 → 0, riporto 1).

**f. Perché  $2^n$  è «1 seguito da n zeri».** In ogni base,  $base^n$  si scrive 1 e n zeri:  $10^3 = 1000$ ,  $2^3 = 1000$  in binario = 8. Per passare all'hex si scrive l'1 **a sinistra**, poi gli n zeri, poi si raggruppa a 4 **da destra**:

$2^{14} = 100\ 0000\ 0000\ 0000 = 0100\ 0000\ 0000\ 0000 = 4000h$

**g. Quanti chip.** Si scrive il numero davanti alla K in binario: **un solo 1** → un chip; più 1 → un chip per ogni 1, se il testo non dà le taglie disponibili.  $16 = 10000 \rightarrow$  un chip ·  $12 = 1100 \rightarrow 8K + 4K \cdot 24 = 11000 \rightarrow 16K + 8K$ . Il docente nell'esercizio 10 sceglie  $8K + 2K + 2K$ , che non è la scomposizione minima.

**Regola di posizionamento:** i pezzi grandi vanno **dalla parte dell'estremo fisso**. In basso si parte da 0000h salendo; in alto si parte da FFFFh scendendo.

12K in alto: RAM 8K E000h–FFFFh (contro la fine)  
RAM 2K D800h–DFFFh  
RAM 2K D000h–D7FFh

× 8K da D000h:  $D000h \div 2000h = 6,5 \rightarrow$  non allineato

## Prova veloce (soluzioni sotto)

1. Bit interni di una EPROM da 256 KB.
2. C000h in binario. Quali bit sono a 1?
3. Fine di un blocco da 256 MB che parte da 00000000h.
4. Un blocco da 1 GB può partire da 60000000h?

Soluzioni

1.  $256\ KB = 2^8 \times 2^{10} = 2^{18} \rightarrow$  **18**.
2.  $1100\ 0000\ 0000\ 0000$ : sono a 1 i due bit più a sinistra (A15 e A14).
3.  $256\ MB = 100000000h \rightarrow$  fine = 0FFFFFFFh.
4.  $1\ GB = 2^{30}$ : servono gli ultimi 30 bit a 0.  $60000000h = 0110\ 0000\ \dots$  ha il bit 29 a 1 → **no**. Partenze valide:  $00000000h, 40000000h, 80000000h, C0000000h$ .